INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

IIIT Hyderabad

ICPC**2

Arya Mahendraker, Kushal Balabhadruni, Sriteja Pashya

2025-12-29

Contents

Stress (1)

check.h

Description: run stress test

d41d8c, 44 lines

```
/*
1. Make in directory

#!/bin/bash

g++ gen.cpp -o gen -O2
g++ brute.cpp -o brute -O2
g++ sol.cpp -o sol -O2

for ((t=1; ; t++)); do
    echo "Test $t"
    ./gen 500 > in/in${t}.txt
    # cat in/in${t}.txt
    ./brute < in/in${t}.txt > out1.txt
    ./sol < in/in${t}.txt > out2.txt

    # diff out1.txt out2.txt

    if ! diff -q out1.txt out2.txt >/dev/null; then
        echo "Mismatch found on test $t!"
        cat in/in${t}.txt
        diff out1.txt out2.txt
        break
    fi
done

also for times, use this:
#ifndef ONLINE_JUDGE
clock_t tStart = clock();
#endif

void runtime() {
#ifndef ONLINE_JUDGE
    double elapsed =
        (double)(clock() - tStart) / CLOCKS_PER_SEC;

    fprintf(stderr,
            ">> Runtime: %.10fs\n",
            elapsed);
#endif
}

*/
```

Contest (2)

template.h

Description: template

<bits/stdc++.h>, <ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/tree_policy.hpp> 981af6, 43 lines

```
using namespace std;
using namespace __gnu_pbds;
typedef tree<int, null_type, less_equal<int>, rb_tree_tag
    ,
             tree_order_statistics_node_update>
    ordered_multiset;
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update>
    ordered_set;
// find_by_order returns iterator to kth largest (0-
    indexed)
// order_of_key returns number of elements strictly less
    than given value -->
// basically index (0-indexed) for multiset, to erase use
     upper_bound.
// Upper_bound lower_bound exchange their roles
#define ONLINE_JUDGE
#ifndef ONLINE_JUDGE
#define db(x) cerr << #x << " == " << x << endl
#define dbs(x) cerr << x << endl
#else
#define db(x) ((void)0)
#define dbs(x) ((void)0)
#endif

#define int long long
#define fast()

    \
  ios_base::sync_with_stdio(0);
                                                    \
  cin.tie(NULL);

      \
  cout.tie(NULL);
#define fr(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define pb push_back
#define prDouble(x) cout << fixed << setprecision(10) <<
    x
int M = 1e9 + 7;
#define all(x) x.begin(), x.end()
#define allr(x) x.rbegin(), x.rend()
#define sz(x) (int)x.size()
void solve() {}
signed main() {
  fast();
  int t = 1;
  cin >> t;
  while (t--) {
    solve();
  }
  return 0;
}
```

rng.h

Description: rng

e7e903, 10 lines

```
mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());

int random(int a, int b) {
  if (a > b)
    return 0;
  return a + rng() % (b - a + 1);
}
double random_double(double a, double b) {
  return a + (b - a) * (rng() / (double)rng.max());
}
```

troubleshoot.txt

Data structures (3)

SegTree.h

Description: SegTree

b79e63, 23 lines

```
struct segtree {
  typedef int T;
  // for max segtree, set unit = INT_MIN and f(a,b) = max
      (a,b)
  static constexpr T unit = 0;     // identity for sum
  T f(T a, T b) { return a + b; } // (any associative fn)
  vector<T> s;
  int n;
  segtree(int n = 0, T def = unit) : s(2 * n, def), n(n)
      {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2)
        ra = f(ra, s[b++]);
      if (e % 2)
        rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

LazySegTree.h

Description: LazySegTree

9376a1, 97 lines

```
template <typename T, typename U> struct seg_tree_lazy {
  int S, H;
  T zero;
  vector<T> value;
  U noop;
  vector<bool> dirty;
  vector<U> prop;
  seg_tree_lazy(int _S, T _zero = T(), U _noop = U()) {
    zero = _zero, noop = _noop;
    for (S = 1, H = 1; S < _S;)
      S *= 2, H++;
    value.resize(2 * S, zero);
    dirty.resize(2 * S, false);
    prop.resize(2 * S, noop);
  }
  void set_leaves(vector<T> &leaves) {
    copy(leaves.begin(), leaves.end(), value.begin() + S)
        ;
    for (int i = S - 1; i > 0; i--)
      value[i] = value[2 * i] + value[2 * i + 1];
  }
  void apply(int i, U &update) {
    value[i] = update(value[i]);
    if (i < S) {
      prop[i] = prop[i] + update;
      dirty[i] = true;
    }
  }
  void rebuild(int i) {
    for (int l = i / 2; l; l /= 2) {
      T combined = value[2 * l] + value[2 * l + 1];
      value[l] = prop[l](combined);
    }
  }
  void propagate(int i) {
    for (int h = H; h > 0; h--) {
      int l = i >> h;
      if (dirty[l]) {
        apply(2 * l, prop[l]);
        apply(2 * l + 1, prop[l]);
        prop[l] = noop;
        dirty[l] = false;
      }
```

```
    }
  }
  void upd(int i, int j, U update) {
    i += S, j += S;
    propagate(i), propagate(j);
    for (int l = i, r = j; l <= r; l /= 2, r /= 2) {
      if ((l & 1) == 1)
        apply(l++, update);
      if ((r & 1) == 0)
        apply(r--, update);
    }
    rebuild(i), rebuild(j);
  }
  T query(int i, int j) {
    i += S, j += S;
    propagate(i), propagate(j);
    T res_left = zero, res_right = zero;
    for (; i <= j; i /= 2, j /= 2) {
      if ((i & 1) == 1)
        res_left = res_left + value[i++];
      if ((j & 1) == 0)
        res_right = value[j--] + res_right;
    }
    return res_left + res_right;
  }
};
struct node {
  int sum, width;
  node operator+(const node &n) {
    // change 1
    return {sum + n.sum, width + n.width};
  }
};
struct update {
  bool type; // 0 for add, 1 for reset
  int value;
  node operator()(const node &n) {
    // change 2
    if (type)
      return {n.width * value, n.width};
    else
      return {n.sum + n.width * value, n.width};
  }
  update operator+(const update &u) {
    // change 3
    if (u.type)
      return u;
    return {type, value + u.value};
  }
};
// Example Initialization
// seg_tree_lazy<node, update> lst(size, {0, 1}, {0, 0});
// lst.set_leaves(leaves);
// lst.upd(l, r, {0, value});
// auto result = lst.query(l, r);
```

DSU.h

Description: DSU

46e4c4, 23 lines

```
struct DSU {
  int n;
  vi parent;
  vi size;
  DSU(int _n) : n(_n), parent(n), size(n, 1) { iota(
      parent.begin(), parent.end(), 0); }
  int find_set(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find_set(parent[x]);
  }
  int getSize(int x) { return size[find_set(x)]; }  //
      returns size of component of x
  void union_sets(int x, int y) {
    x = find_set(x);
    y = find_set(y);
    if (x == y) return;
```

```
    if (size[x] > size[y]) {
      parent[y] = x;
      size[x] += size[y];
    } else {
      parent[x] = y;
      size[y] += size[x];
    }
  }
};
```

RMQ.h

Description: RMQ

7bc356, 16 lines

```
template <class T>
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k
        ) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      for (int j = 0; j < sz(jmp[k]); j++)
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw
            ]);
    }
  }
  T query(int a, int b) {
    assert(a <= b);  // tie(a, b) = minimax(a, b)
    int dep = 63 - __builtin_clzll(b - a + 1);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep) + 1])
        ;
  }
};
```

Treap.h

Description: Treap

1048b8, 68 lines

```
/*A short self-balancing tree. It acts as a
sequential container with log-time splits/joins, and
is easy to augment with additional data.
Time: $O(\log N)$*/

struct Node {
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  Node(int val) : val(val), y(rand()) {}
  void recalc();
};

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

template <class F>
void each(Node* n, F f) {
  if (n) {
    each(n->l, f);
    f(n->val);
    each(n->r, f);
  }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  if (cnt(n->l) >= k) {  // "n->val >= k" for lower_bound
      (k)
    auto pa = split(n->l, k);
    n->l = pa.second;
    n->recalc();
    return {pa.first, n};
  } else {
    auto pa = split(n->r, k - cnt(n->l) - 1);  // and
        just "k"
    n->r = pa.first;
    n->recalc();
    return {n, pa.second};
  }
```

```
}

Node* merge(Node* l, Node* r) {
  if (!l) return r;
  if (!r) return l;
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    l->recalc();
    return l;
  } else {
    r->l = merge(l, r->l);
    r->recalc();
    return r;
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto [l, r] = split(t, pos);
  return merge(merge(l, n), r);
}

// Example application: move the range [l, r) to index k
void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c;
  tie(a, b) = split(t, l);
  tie(b, c) = split(b, r - l);
  if (k <= l)
    t = merge(ins(a, b, k), c);
  else
    t = merge(a, ins(c, b, k - r));
}
```

cht.h

Description: cht

b46218, 33 lines

```
struct Line {
  mutable i64 m, c, p;
  bool operator<(const Line& o) const { return m < o.m; }
  bool operator<(i64 x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const i64 inf = LONG_LONG_MAX;
  i64 div(i64 a, i64 b) {  // floored division
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->m == y->m)
      x->p = x->c > y->c ? inf : -inf;
    else
      x->p = div(y->c - x->c, x->m - y->m);
    return x->p >= y->p;
  }
  void add(i64 m, i64 c) {
    auto z = insert({m, c, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase
      (y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  i64 query(i64 x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.m * x + l.c;
  }
};
```

Mos.h

Description: Mos

1cb2c0, 38 lines

```
int BLOCK = DO_NOT_FORGET_TO_CHANGE_THIS;
struct Query {
  int l, r, id;
```

```
  Query(int _l, int _r, int _id) : l(_l), r(_r), id(_id)
     {}
  bool operator<(Query &o) {
    int mblock = l / BLOCK, oblock = o.l / BLOCK;
    return (mblock < oblock) or
           (mblock == oblock and mblock % 2 == 0 and r <
              o.r) or
           (mblock == oblock and mblock % 2 == 1 and r >
              o.r);
  };
};
void solve() {
  vector<Query> queries;
  queries.reserve(q);
  for (int i = 0; i < q; i++) {
    int l, r;
    cin >> l >> r;
    l--, r--;
    queries.emplace_back(l, r, i);
  }
  sort(all(queries));
  int ans = 0;
  auto add = [&](int v) {};
  auto rem = [&](int v) {};
  vector<int> out(q); // Change out type if necessary
  int cur_l = 0, cur_r = -1;
  for (auto &[l, r, id] : queries) {
    while (cur_l > l)
      add(--cur_l);
    while (cur_l < l)
      rem(cur_l++);
    while (cur_r < r)
      add(++cur_r);
    while (cur_r > r)
      rem(cur_r--);
    out[id] = ans;
  }
}
```

MosUpdate.h

Description: Mos with Updates

9bb5d0, 75 lines

```
// Set BLOCK_SIZE to N^(2/3) for Mo's with updates
int BLOCK_SIZE = CHANGE;

struct Query {
    int l, r, t, id;
    bool operator<(const Query& o) const {
        int l_block = l / BLOCK_SIZE;
        int o_l_block = o.l / BLOCK_SIZE;
        if (l_block != o_l_block) return l_block <
           o_l_block;
        int r_block = r / BLOCK_SIZE;
        int o_r_block = o.r / BLOCK_SIZE;
        if (r_block != o_r_block) return r_block <
           o_r_block;
        return t < o.t;
    }
};

struct Update {
    int pos, prev_val, new_val;
};

void solve_mo_with_updates() {
    // BLOCK_SIZE = pow(n, 2.0/3.0);

    vector<Query> queries;
    vector<Update> updates;
    // for (int i = 0; i < q; i++) {
    //     int type; cin >> type;
    //     if (type == 1) { // Query
    //         int l, r; cin >> l >> r; l--, r--;
    //         queries.push_back({l, r, (int)updates.size
       (), (int)queries.size()});
```

```
//      } else { // Update
//          int pos, val; cin >> pos >> val; pos--;
//          updates.push_back({pos, 0, val}); //
    prev_val can be filled later
//      }
// }

sort(all(queries));

int ans = 0;
auto add = [&](int idx) {};
auto rem = [&](int idx) {};
auto apply_update = [&](int update_idx, int current_l
    , int current_r) {
    auto& upd = updates[update_idx];
    // If update position is within current range [l,
        r], adjust answer
    if (upd.pos >= current_l && upd.pos <= current_r)
        {
        rem(upd.pos);
        a[upd.pos] = upd.new_val;
        add(upd.pos);
    } else {
        a[upd.pos] = upd.new_val;
    }
};
auto undo_update = [&](int update_idx, int current_l,
     int current_r) {
    auto& upd = updates[update_idx];
    if (upd.pos >= current_l && upd.pos <= current_r)
        {
        rem(upd.pos);
        a[upd.pos] = upd.prev_val;
        add(upd.pos);
    } else {
        a[upd.pos] = upd.prev_val;
    }
};

vector<int> out(queries.size());
int cur_l = 0, cur_r = -1, cur_t = 0;
for (auto& [l, r, t, id] : queries) {
    while (cur_t < t) apply_update(cur_t++, cur_l,
        cur_r);
    while (cur_t > t) undo_update(--cur_t, cur_l,
        cur_r);
    while (cur_l > l) add(--cur_l);
    while (cur_r < r) add(++cur_r);
    while (cur_l < l) rem(cur_l++);
    while (cur_r > r) rem(cur_r--);
    out[id] = ans;
}
}
```

RangeUpdateTree.h

Description: Range update tree for range updates and point queries

Time: $\mathcal{O}(\log N)$

a653e9, 32 lines

```
template <typename T, typename F> struct RangeUpdateTree
    {
  int n;
  vector<T> tree;
  T identity;
  F merge;
  RangeUpdateTree(const vector<T> &arr, T id, F _m)
      : n((int)arr.size()), tree(2 * n), identity(id),
        merge(_m) {
    for (int i = 0; i < n; i++)
      tree[n + i] = arr[i];
    for (int i = n - 1; i >= 1; i--)
      tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
  }
  void update(int l, int r, T value) {
    assert(l >= 0 && r < n && l <= r);
    for (l += n, r += n; l <= r; l >>= 1, r >>= 1) {
```

```
      if (l & 1)
        tree[l] = merge(value, tree[l]), l++;
      if (!(r & 1))
        tree[r] = merge(value, tree[r]), r--;
    }
    if (l == r)
      tree[l] = merge(value, tree[l]);
  }
  T query(int v) {
    T res = tree[v += n];
    for (; v > 1; v >>= 1)
      res = merge(res, tree[v >> 1]);
    return res;
  }
};
// ex: RangeUpdateTree<int, decltype(join)> v(vi (n, 1e9)
    , 1e9, join); use auto
// func for join
```

segtreeFaisal.h

Description: LazySegTree

282af9, 49 lines

```
template <typename T, typename F>
struct SegTree {
  int n, off, ct;
  vector<T> t;
  const T id;
  F f;
  SegTree(const vector<T>& a, T _id, F _f)
      : n(sz(a)), off(1 << 32 - __builtin_clz(n)), ct(n ^
          off >> 1), t(2 * n), id(_id), f(_f) {
    for (int i = 0; i < 2 * ct; i++) t[off + i] = a[i];
    for (int i = 2 * ct; i < n; i++) t[i + off - n] = a[i
        ];
    for (int i = n - 1; i >= 1; i--) t[i] = f(t[2 * i], t
        [2 * i + 1]);
  }
int i2leaf(int i) { return i + off - (i < 2 * ct ? 0 :
    n); }
int leaf2i(int l) { return l - off + (l < off ? n : 0);
    }
T query(int l, int r) {
  l = (l < 2 * ct) ? (l + off) : 2 * (l + off - n);
  r = (r < 2 * ct) ? (r + off) : 2 * (r + off - n);
  r += (r >= 2 * n);
  T resl(id), resr(id);
  for (; l <= r; l >>= 1, r >>= 1) {
    if (l == r) {
      resl = f(resl, t[l]);
      break;
    }
    if (l & 1) resl = f(resl, t[l++]);
    if (!(r & 1)) resr = f(t[r--], resr);
  }
  return f(resl, resr);
}
void update(int v, T value) {
  for (t[v = i2leaf(v)] = value; v >>= 1;)
    t[v] = f(t[2 * v], t[2 * v + 1]);
}
int lower_bound(int k) {
  if (t[1] < k) return n;
  T rem = id;
  int v = 1;
  while (v < n) {
    T resl = f(rem, t[2 * v]);
    if (resl >= k) {
      v = 2 * v;
    } else {
      rem = resl;
      v = 2 * v + 1;
    }
  }
  return leaf2i(v);
```

```
  }
};
```

mergesorttree1.h

Description: Merge sort tree 1, for Static Count of Lesser Elements range queries.

<bits/stdc++.h> 25ca75, 55 lines

```
// We are given an array a1,a2,a3...an and an integer q.
// The i_th of the next q lines has three integers li,ri,
   xi.
// The answer for this query is how many aj such that li
   <= j <= ri and aj<x.
using namespace std;
const int N=1e5+5;
int n,q,a[N];
vector<int> seg[4*N+5];
//build merge sort tree
void build(int node,int l,int r){
    //if range length is 1 there's only one element to
       add and no children
    if (l==r){
        seg[node].push_back(a[l]);
        return;
    }int mid=(l+r)/2;
    build(node*2,l,mid);
    build(node*2+1,mid+1,r);
    int i=0,j=0;
    // use two pointers to merge the two vectors in O(r-l
       +1)
    while (i<seg[node*2].size() && j<seg[node*2+1].size()
       ){
        if (seg[node*2][i]<seg[node*2+1][j]) seg[node].
           push_back(seg[node*2][i++]);
        else seg[node].push_back(seg[node*2+1][j++]);
    }
    while (i<seg[node*2].size()) seg[node].push_back(seg[
       node*2][i++]);
    while (j<seg[node*2+1].size()) seg[node].push_back(
       seg[node*2+1][j++]);
    return;
}
//query
int query(int node,int l,int r,int lx,int rx,int x){
    //if outside -> 0
    if (l>rx || r<lx) return 0;
    //if inside do binary search
    if (l>=lx && r<=rx){
        int L=0,R=seg[node].size()-1,mid,ans=0;
        while (L<=R){
            mid=(L+R)/2;
            if (seg[node][mid]<x){
                ans=mid+1;
                L=mid+1;
            }else R=mid-1;
        }return ans;
    }
    int mid=(l+r)/2;
    return query(node*2,l,mid,lx,rx,x)+query(node*2+1,mid
       +1,r,lx,rx,x);
}
signed main(){
    cin>>n>>q;
    for (int i=1;i<=n;i++) cin>>a[i];
    build(1,1,n);
    while (q--){
        int l,r,x;
        cin>>l>>r>>x;
        cout<<query(1,1,n,l,r,x)<<endl;
    }
    return 0;
}
```

mergesorttree2.h

Description: Merge sort tree 2, for Dynamic Lower Bound problem.

<bits/stdc++.h> a45602, 58 lines

```
// given array and updates,
// should find lower bound of x in range l to r.

using namespace std;
const int N=1e5+5;
int n,q,a[N];
multiset<int> seg[4*N+5];
void build(int node,int l,int r){
    if (l==r){
        seg[node].insert(a[l]);
        return;
    }int mid=(l+r)/2;
    build(node*2,l,mid);
    build(node*2+1,mid+1,r);
    for (int i=l;i<=r;i++) seg[node].insert(a[i]);
    return;
}
void edit(int node,int l,int r,int idx,int val){
    if (l==r){
        seg[node].erase(a[idx]);
        seg[node].insert(val);
        return;
    }int mid=(l+r)/2;
    if (idx<=mid) edit(node*2,l,mid,idx,val);
    else edit(node*2+1,mid+1,r,idx,val);
    seg[node].erase(a[idx]);
    seg[node].insert(val);
    return;
}
int query(int node,int l,int r,int lx,int rx,int x){
    if (l>rx || r<lx) return INT_MAX;
    if (l>=lx && r<=rx){
        auto it=seg[node].lower_bound(x);
        if (it==seg[node].end()) return INT_MAX;
        return *it;
    }int mid=(l+r)/2;
    return min(query(node*2,l,mid,lx,rx,x),query(node
        *2+1,mid+1,r,lx,rx,x));
}
int32_t main(){
    ios_base::sync_with_stdio(false);cin.tie(NULL);cout.
        tie(NULL);
    cin>>n>>q;
    for (int i=1;i<=n;i++) cin>>a[i];
    build(1,1,n);
    while (q--){
        bool t;cin>>t;
        if (!t){
            int idx,val;
            cin>>idx>>val;
            edit(1,1,n,idx,val);
            a[idx]=val;
            continue;
        }int l,r,x;cin>>l>>r>>x;
        int y=query(1,1,n,l,r,x);
        if (y==INT_MAX) cout<<-1<<endl;
        else cout<<y<<endl;
    }
    return 0;
}
```

Graph (4)

2sat.h

Description: 2sat

386ce8, 75 lines

```
/*
ts.either(x, y);
ts.either(~x, ~y); these two do x xor y
```

```
ts.setValue(x, x); assert x is true

use ~x to denote not x

call ts.solve() to run the solver, returns if a solution
    exists
if exists: ts.values[i] contains the assignments
*/

struct TwoSat {
  int N;
  vector<vi> gr;
  vi values;  // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2 * n) {}

  int addVar() {  // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2 * f, -1 - 2 * f);
    j = max(2 * j, -1 - 2 * j);
    gr[f].push_back(j ^ 1);
    gr[j].push_back(f ^ 1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) {  // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    for (int i = 2; i < sz(li); i++) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z;
  int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x;
    z.push_back(i);
    for (int e : gr[i])
      if (!comp[e])
        low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
        x = z.back();
        z.pop_back();
        comp[x] = low;
        if (values[x >> 1] == -1)
          values[x >> 1] = x & 1;
      } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2 * N, 0);
    comp = val;
    for (int i = 0; i < 2 * N; i++)
      if (!comp[i]) dfs(i);
    for (int i = 0; i < N; i++)
      if (comp[2 * i] == comp[2 * i + 1]) return 0;
    return 1;
  }
};
```

bellman.h

Description: bellman

d7cfb8, 27 lines

```
bool bellman_ford(int n, int src, vector<vector<pair<int,
    int>>> &adj,
                  vector<long long> &dist) {
  const long long INF = 1e18;
  dist.assign(n, INF);
  dist[src] = 0;
  for (int i = 0; i < n - 1; i++) {
    for (int u = 0; u < n; u++) {
      if (dist[u] == INF)
        continue;
      for (auto &p : adj[u]) {
        int v = p.first, w = p.second;
        if (dist[u] + w < dist[v])
          dist[v] = dist[u] + w;
      }
    }
  }
  for (int u = 0; u < n; u++) {
    if (dist[u] == INF)
      continue;
    for (auto &p : adj[u]) {
      int v = p.first, w = p.second;
      if (dist[u] + w < dist[v])
        return false;
    }
  }
  return true;
}
```

bridges.h

Description: bridges

bfead9, 106 lines

```
int n;                        // number of nodes
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
  visited[v] = true;
  tin[v] = low[v] = timer++;
  for (int to : adj[v]) {
    if (to == p)
      continue;
    if (visited[to]) {
      low[v] = min(low[v], tin[to]);
    } else {
      dfs(to, v);
      low[v] = min(low[v], low[to]);
      if (low[to] > tin[v])
        IS_BRIDGE(v, to);
    }
  }
}

void find_bridges() {
  timer = 0;
  visited.assign(n, false);
  tin.assign(n, -1);
  low.assign(n, -1);
  for (int i = 0; i < n; ++i) {
    if (!visited[i])
      dfs(i);
  }
}

// ARTICULATION POINTS:
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
```

```
int timer;
void dfs(int v, int p = -1) {
  visited[v] = true;
  tin[v] = low[v] = timer++;
  int children = 0;
  for (int to : adj[v]) {
    if (to == p)
      continue;
    if (visited[to]) {
      low[v] = min(low[v], tin[to]);
    } else {
      dfs(to, v);
      low[v] = min(low[v], low[to]);
      if (low[to] >= tin[v] && p != -1)
        IS_CUTPOINT(v);
      ++children;
    }
  }
  if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}
void find_cutpoints() {
  timer = 0;
  visited.assign(n, false);
  tin.assign(n, -1);
  low.assign(n, -1);
  for (int i = 0; i < n; ++i) {
    if (!visited[i])
      dfs(i);
  }
}

// arya bridges
void findBridges_dfs(int u, int p, int &time, vector<
    vector<int>> &adj,
                     vector<int> &disc, vector<int> &low,
                     vector<pair<int, int>> &bridges) {
  disc[u] = low[u] = time++;

  for (int v : adj[u]) {
    if (v == p)
      continue;

    if (disc[v] != -1) {
      low[u] = min(low[u], disc[v]);
    } else {
      findBridges_dfs(v, u, time, adj, disc, low, bridges
          );
      low[u] = min(low[u], low[v]);
      if (low[v] > disc[u]) {
        bridges.push_back({u, v});
      }
    }
  }
}

vector<pair<int, int>> findBridges(int n, vector<vector<
    int>> &adj) {
  vector<int> disc(n, -1), low(n, -1);
  vector<pair<int, int>> bridges;
  int time = 0;

  for (int i = 0; i < n; ++i) {
    if (disc[i] == -1) {
      findBridges_dfs(i, -1, time, adj, disc, low,
          bridges);
    }
  }
  return bridges;
}
```

dijkstra.h

Description: dijkstra

66994b, 32 lines

```
const int INF = 1000000000;
```

```
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> &d, vector<int> &p) {
  int n = adj.size();
  d.assign(n, INF);
  p.assign(n, -1);
  vector<bool> u(n, false);

  d[s] = 0;
  for (int i = 0; i < n; i++) {
    int v = -1;
    for (int j = 0; j < n; j++) {
      if (!u[j] && (v == -1 || d[j] < d[v]))
        v = j;
    }

    if (d[v] == INF)
      break;

    u[v] = true;
    for (auto edge : adj[v]) {
      int to = edge.first;
      int len = edge.second;

      if (d[v] + len < d[to]) {
        d[to] = d[v] + len;
        p[to] = v;
      }
    }
  }
}
```

Dinic.h

Description: Dinic

00944d, 49 lines

```
// Flow algorithm with complexity $O(VE\log U)$ where $U
    = \max |\text{cap}|$.
// $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)
    $ for bipartite matching.
using ll = long long;
#define rep(i, j, k) for (int i = j; i < k; i++)
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); }  // if you
        need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0;
    q[0] = s;
    rep(L, 0, 31) do {  // 'int L=30' maybe faster for
        random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
```

```
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    }
    while (lvl[t])
      ;
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{
     e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (
     optional)
```

```
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][
        cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[
        i];
  return ret;
}
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src
    =0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x])
        ;
    if (it == end){ ret.push_back(x); s.pop_back();
        continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1)
      return {};
  return {ret.rbegin(), ret.rend()};
}
```

FloydWarshall.h

Description: FloydWarshall

8dfa30, 14 lines

```
const long long INF = (long long)1e18;
bool floyd_warshall(int n, vector<vector<long long>> &
    dist) {
  // initialise dist with edge weights, INF if no edge
      exists
  for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (dist[i][k] < INF && dist[k][j] < INF)
          dist[i][j] = min(dist[i][j], dist[i][k] + dist[
              k][j]);

  for (int i = 0; i < n; i++)
    if (dist[i][i] < 0)
      return true;
  return false;
}
```

HLD.h

Description: HLD

7f333e, 55 lines

```
struct HLD {
  int n, timer = 0;
  vi top, tin, p, sub;
  HLD(vvi &adj) : n(sz(adj)), top(n), tin(n), p(n, -1),
      sub(n, 1) {
    vi ord(n + 1);
    for (int i = 0, t = 0, v = ord[i]; i < n; v = ord[++i
        ])
      for (auto &to : adj[v])
        if (to != p[v])
          p[to] = v, ord[++t] = to;
    for (int i = n - 1, v = ord[i]; i > 0; v = ord[--i])
      sub[p[v]] += sub[v];
    for (int v = 0; v < n; v++)
      if (sz(adj[v]))
        iter_swap(begin(adj[v]), max_element(all(adj[v]),
            [&](int a, int b) {
                  return make_pair(a != p[v], sub[a]) <
                         make_pair(b != p[v], sub[b]);
                }));
    function<void(int)> dfs = [&](int v) {
      tin[v] = timer++;
      for (auto &to : adj[v])
        if (to != p[v]) {
          top[to] = (to == adj[v][0] ? top[v] : to);
          dfs(to);
        }
    };
    dfs(0);
  }
  int lca(int u, int v) {
    return process(u, v, [](int, int) {});
  }
  template <class B> int process(int a, int b, B op, bool
       ignore_lca = false) {
    for (int v;; op(tin[v], tin[b]), b = p[v]) {
      if (tin[a] > tin[b])
        swap(a, b);
      if ((v = top[b]) == top[a])
        break;
    }
    if (int l = tin[a] + ignore_lca, r = tin[b]; l <= r)
      op(l, r);
    return a;
  }
  template <class B> void subtree(int v, B op, bool
      ignore_lca = false) {
    if (sub[v] > 1 or !ignore_lca)
      op(tin[v] + ignore_lca, tin[v] + sub[v] - 1);
  }
```

```
  template <class B>
  void go_up(int node, int anc, B op) {
      while(top[node] != top[anc]) {
          op(tin[top[node]], tin[node]);
          node = p[top[node]];
      }
      op(tin[anc], tin[node]);
  }
};
```

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph *g* should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); hopcroftKarp(g, btoa);`
Time: $\mathcal{O}\left(\sqrt{V}E\right)$

f612e4, 42 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi
    & B) {
  if (A[a] != L) return 0;
  A[a] = -1;
  for (int b : g[a]) if (B[b] == L + 1) {
    B[b] = 0;
    if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A,
        B))
      return btoa[b] = a, 1;
  }
  return 0;
}

int hopcroftKarp(vector<vi>& g, vi& btoa) {
  int res = 0;
  vi A(g.size()), B(btoa.size()), cur, next;
  for (;;) {
    fill(all(A), 0);
    fill(all(B), 0);
    cur.clear();
    for (int a : btoa) if(a != -1) A[a] = -1;
    rep(a,0,sz(g)) if(A[a] == 0) cur.push_back(a);
    for (int lay = 1;; lay++) {
      bool islast = 0;
      next.clear();
      for (int a : cur) for (int b : g[a]) {
        if (btoa[b] == -1) {
          B[b] = lay;
          islast = 1;
        }
        else if (btoa[b] != a && !B[b]) {
          B[b] = lay;
          next.push_back(btoa[b]);
        }
      }
      if (islast) break;
      if (next.empty()) return res;
      for (int a : next) A[a] = lay;
      cur.swap(next);
    }
    rep(a,0,sz(g))
      res += dfs(a, 0, g, btoa, A, B);
  }
}
```

KthAnc.h

Description: KthAnc

28228d, 60 lines

```
// O(log n) LCA with Kth anc
struct LCA {
  int n;
  vvi &adjLists;
  int lg;
  vvi up;
  vi depth;
  LCA(vvi &_adjLists, int root = 0) : n(sz(_adjLists)),
      adjLists(_adjLists) {
    lg = 1;
```

```
    int pw = 1;
    while (pw <= n)
      pw <<= 1, lg++;
    // lg = 20
    up = vvi(n, vi(lg));
    depth.assign(n, -1);
    function<void(int, int)> parentDFS = [&](int from,
        int parent) {
      depth[from] = depth[parent] + 1;
      up[from][0] = parent;
      for (auto to : adjLists[from]) {
        if (to == parent)
          continue;
        parentDFS(to, from);
      }
    };
    parentDFS(root, root);
    for (int j = 1; j < lg; j++) {
      for (int i = 0; i < n; i++) {
        up[i][j] = up[up[i][j - 1]][j - 1];
      }
    }
  }
  int kthAnc(int v, int k) {
    int ret = v;
    int pw = 0;
    while (k) {
      if (k & 1)
        ret = up[ret][pw];
      k >>= 1;
      pw++;
    }
    return ret;
  }
  int lca(int u, int v) {
    if (depth[u] > depth[v])
      swap(u, v);
    v = kthAnc(v, depth[v] - depth[u]);
    if (u == v)
      return v;
    while (up[u][0] != up[v][0]) {
      int i = 0;
      for (; i < lg - 1; i++) {
        if (up[u][i + 1] == up[v][i + 1])
          break;
      }
      u = up[u][i], v = up[v][i];
    }
    return up[u][0];
  };
  int dist(int u, int v) { return depth[u] + depth[v] - 2
      * depth[lca(u, v)]; }
};
```

LCA.h

Description: LCA

993da6, 27 lines

```
// O(1) LCA
struct LCA {
  int T = 0;
  vi st, path, ret;
  vi en, d;
  RMQ<int> rmq;
  LCA(vector<vi> &C)
      : st(sz(C)), en(sz(C)), d(sz(C)), rmq((dfs(C, 0,
        -1), ret)) {}
  void dfs(vvi &adj, int v, int par) {
    st[v] = T++;
    for (auto to : adj[v])
      if (to != par) {
        path.pb(v), ret.pb(st[v]);
        d[to] = d[v] + 1;
        dfs(adj, to, v);
      }
    en[v] = T - 1;
```

```
  }
  bool anc(int p, int c) { return st[p] <= st[c] and en[p
      ] >= en[c]; }
  int lca(int a, int b) {
    if (a == b)
      return a;
    tie(a, b) = minmax(st[a], st[b]);
    return path[rmq.query(a, b - 1)];
  }
  int dist(int a, int b) { return d[a] + d[b] - 2 * d[lca
      (a, b)]; }
};
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i
        ];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; }
        );
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({
          v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) +
            1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old
      (S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

MinCostMaxFlow.h

Description: MinCostMaxFlow

9a3690, 179 lines

```
template <const int MAX_N, typename flow_t,
          typename cost_t, flow_t FLOW_INF,
          cost_t COST_INF, const int SCALE = 16>
struct CostScalingMCMF {
#define sz(a) a.size()
#define zero_stl(v, sz) fill(v.begin(), v.begin() + (sz),
    0)
  struct Edge {
    int v;
    flow_t c;
    cost_t d;
    int r;
    Edge() = default;
    Edge(int v, flow_t c, cost_t d, int r) : v(v), c(c),
       d(d), r(r) {}
  };
  vector<Edge> g[MAX_N];
  cost_t negativeSelfLoop;
  array<cost_t, MAX_N> pi, excess;
  array<int, MAX_N> level, ptr;
  CostScalingMCMF() { negativeSelfLoop = 0; }
  void clear() {
    negativeSelfLoop = 0;
    for (int i = 0; i < MAX_N; i++) g[i].clear();
  }
  void addEdge(int s, int e, flow_t cap, cost_t cost) {
    if (s == e) {
      if (cost < 0) negativeSelfLoop += cap * cost;
      return;
    }
    g[s].push_back(Edge(e, cap, cost, sz(g[e])));
    g[e].push_back(Edge(s, 0, -cost, sz(g[s]) - 1));
  }
  flow_t getMaxFlow(int V, int S, int T) {
    auto BFS = [&]() {
      zero_stl(level, V);
      queue<int> q;
      q.push(S);
      level[S] = 1;
      for (q.push(S); !q.empty(); q.pop()) {
        int v = q.front();
        for (const auto &e : g[v])
          if (!level[e.v] && e.c) q.push(e.v), level[e.v]
              = level[v] + 1;
      }
      return level[T];
    };
    function<flow_t(int, flow_t)> DFS = [&](int v, flow_t
        fl) {
      if (v == T || fl == 0) return fl;
      for (int &i = ptr[v]; i < (int)g[v].size(); i++) {
        Edge &e = g[v][i];
        if (level[e.v] != level[v] + 1 || !e.c) continue;
        flow_t delta = DFS(e.v, min(fl, e.c));
        if (delta) {
          e.c -= delta;
          g[e.v][e.r].c += delta;
          return delta;
        }
      }
      return flow_t(0);
    };
    flow_t maxFlow = 0, tmp = 0;
    while (BFS()) {
      zero_stl(ptr, V);
      while ((tmp = DFS(S, FLOW_INF))) maxFlow += tmp;
    }
```

```
    return maxFlow;
  }
  pair<flow_t, cost_t> maxflow(int N, int S, int T) {
    flow_t maxFlow = 0;
    cost_t eps = 0, minCost = 0;
    stack<int, vector<int>> stk;
    auto c_pi = [&](int v, const Edge &edge) { return
       edge.d + pi[v] - pi[edge.v]; };
    auto push = [&](int v, Edge &edge, flow_t delta, bool
        flag) {
      delta = min(delta, edge.c);
      edge.c -= delta;
      g[edge.v][edge.r].c += delta;
      excess[v] -= delta;
      excess[edge.v] += delta;
      if (flag && 0 < excess[edge.v] && excess[edge.v] <=
          delta) stk.push(edge.v);
    };
    auto relabel = [&](int v, cost_t delta) { pi[v] -=
       delta + eps; };
    auto lookAhead = [&](int v) {
      if (excess[v]) return false;
      cost_t delta = COST_INF;
      for (auto &e : g[v]) {
        if (e.c <= 0) continue;
        cost_t cp = c_pi(v, e);
        if (cp < 0)
          return false;
        else
          delta = min(delta, cp);
      }
      relabel(v, delta);
      return true;
    };
    auto discharge = [&](int v) {
      cost_t delta = COST_INF;
      for (int i = 0; i < sz(g[v]); i++) {
        Edge &e = g[v][i];
        if (e.c <= 0) continue;
        cost_t cp = c_pi(v, e);
        if (cp < 0) {
          if (lookAhead(e.v)) {
            i--;
            continue;
          }
          push(v, e, excess[v], true);
          if (excess[v] == 0) return;
        } else
          delta = min(delta, cp);
      }
      relabel(v, delta);
      stk.push(v);
    };
    zero_stl(pi, N);
    zero_stl(excess, N);
    for (int i = 0; i < N; i++)
      for (auto &e : g[i]) minCost += e.c * e.d, e.d *=
         MAX_N + 1, eps = max(eps, e.d);
    maxFlow = getMaxFlow(N, S, T);
    while (eps > 1) {
      eps /= SCALE;
      if (eps < 1) eps = 1;
      stk = stack<int, vector<int>>();
      for (int v = 0; v < N; v++)
        for (auto &e : g[v])
          if (c_pi(v, e) < 0 && e.c > 0) push(v, e, e.c,
             false);
      for (int v = 0; v < N; v++)
        if (excess[v] > 0) stk.push(v);
      while (stk.size()) {
        int top = stk.top();
        stk.pop();
        discharge(top);
      }
```

```
    }
    for (int v = 0; v < N; v++)
      for (auto &e : g[v]) e.d /= MAX_N + 1, minCost -= e
          .c * e.d;
    minCost = minCost / 2 + negativeSelfLoop;
    return {maxFlow, minCost};
  }
};

void solve() {
  CostScalingMCMF<102, int, int, 100, 100> flow;
  int n, m;
  cin >> n >> m;
  int start = 0;
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      int inp;
      cin >> inp;
      if (inp) {
        flow.addEdge(i + 1, n + 1 + j, 1, 0);
        start++;
      } else
        flow.addEdge(i + 1, n + 1 + j, 1, 1);
    }
  }
  int counta = 0, countb = 0;
  for (int i = 0; i < n; i++) {
    int inp;
    cin >> inp;
    counta += inp;
    flow.addEdge(0, i + 1, inp, 0);
  }
  for (int i = 0; i < m; i++) {
    int inp;
    cin >> inp;
    countb += inp;
    flow.addEdge(n + i + 1, n + m + 1, inp, 0);
  }
  if (counta != countb) {
    cout << -1 << endl;
    return;
  }
  pii t = flow.maxflow(102, 0, n + m + 1);
  if (t.first != counta) {
    cout << -1 << endl;
    return;
  }
  cout << t.second + start + t.second - counta << endl;
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h" da4196, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
```

```
}
```

SCC.h

Description: SCC

d54e21, 30 lines

```
struct SCC {
  int n;
  vi val, cc, z;
  vvi comps;
  SCC(vvi& adj) : n(sz(adj)), val(n), cc(n, -1) {
    int timer = 0;
    function<int(int)> dfs = [&](int x) {
      int low = val[x] = ++timer, b;
      z.push_back(x);
      for (auto y : adj[x])
        if (cc[y] < 0)
          low = min(low, val[y] ?: dfs(y));

      if (low == val[x]) {
        comps.push_back(vi());
        do {
          b = z.back();
          z.pop_back();
          comps.back().push_back(b);
          cc[b] = sz(comps) - 1;
        } while (x != b);
      }
      return val[x] = low;
    };
    for (int i = 0; i < n; i++)
      if (cc[i] < 0) dfs(i);
  }
  int operator[](int i) { return cc[i]; }
  int size(int i) { return sz(comps[cc[i]]); }
};
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}\left(N^2M\right)$

1e0fe9, 31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
  if (a.empty()) return {0, {}};
  int n = sz(a) + 1, m = sz(a[0]) + 1;
  vi u(n), v(m), p(m), ans(n - 1);
  rep(i,1,n) {
    p[0] = i;
    int j0 = 0; // add "dummy" worker 0
    vi dist(m, INT_MAX), pre(m, -1);
    vector<bool> done(m + 1);
    do { // dijkstra
      done[j0] = true;
      int i0 = p[j0], j1, delta = INT_MAX;
      rep(j,1,m) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;
      }
      rep(j,0,m) {
        if (done[j]) u[p[j]] += delta, v[j] -= delta;
        else dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
```

```
}
```

Number theory (5)

ModularArithmetic.h

Description: ModularArithmetic

778e0b, 122 lines

```
int add(int x, int y, int m = M) {
  int ret = (x + y) % m;
  if (ret < 0)
    ret += m;
  return ret;
}
int mult(int x, int y, int m = M) {
  int ret = (x * y) % m;
  if (ret < 0)
    ret += m;
  return ret;
}
int pw(int a, int b, int m = M) {
  int ret = 1;
  int p = a;
  while (b) {
    if (b & 1)
      ret = mult(ret, p, m);
    b >>= 1;
    p = mult(p, p, m);
  }
  return ret;
}

#define LL int
const long long mod = M;

int euclid(int a, int b, int &x, int &y) {
  if (!b)
    return x = 1, y = 0, a;
  int d = euclid(b, a % b, y, x);
  return y -= a / b * x, d;
}

int modulo_inverse(int a, int m) {
  int x, y;
  int g = euclid(a, m, x, y);
  if (g != 1) {
    return -1;
  } else {
    x = (x % m + m) % m;
    return x;
  }
}

LL mod_mul(LL a, LL b) {
  a = a % mod;
  b = b % mod;
  return (((a * b) % mod) + mod) % mod;
}

LL mod_add(LL a, LL b) {
  a = a % mod;
  b = b % mod;
  return (((a + b) % mod) + mod) % mod;
}

const int MX = 5e5 + 1;
vector<int> inv(MX + 1), fci(MX + 1), fc(MX + 1);
const int Mod = 1e9 + 7;

void Inverses() {
  inv[1] = 1;
  for (int i = 2; i <= MX; i++) {
    inv[i] = Mod - Mod / i * inv[Mod % i] % Mod;
  }
```

```
}

void Factorials() {
  fc[0] = fc[1] = 1;
  for (int i = 2; i <= MX; i++) {
    fc[i] = fc[i - 1] * i % Mod;
  }
}

void InverseFactorials() {
  Inverses();
  Factorials();
  fci[1] = fci[0] = 1;
  for (int i = 2; i <= MX; i++) {
    fci[i] = fci[i - 1] * inv[i] % Mod;
  }
}

int nck(int num, int k) {
  if (num < 0) {
    return 0;
  }
  if (k < 0) {
    return 0;
  }
  if (num < k) {
    return 0;
  } else {
    return fc[num] * fci[k] % Mod * fci[num - k] % Mod;
  }
}

int BinExpItermod(int a, int b) {
  int ans = 1;
  while (b > 0) {
    if (b & 1) {
      ans = (ans * a) % mod;
    }
    a = (a * a) % mod;
    b = b >> 1;
  }
  return ans;
}

int gcd(int a, int b) {
    if (a == 0)
        return b;
    return gcd(b % a, a);
}

int lcm(int a, int b) {
    int gc = gcd(a, b);
    int ans = a * b;
    ans /= gc;
    return ans;
}
```

MillerRabin.h

Description: MillerRabin

71e4cd, 22 lines

```
u64 mult(u64 a, u64 b, u64 m = M) {
  i64 ret = a * b - m * (u64)(1.L / m * a * b);
  return ret + m * (ret < 0) - m * (ret >= (i64)m);
}
u64 pw(u64 b, u64 e, u64 m = M) {
  u64 ret = 1;
  for (; e; b = mult(b, b, m), e >>= 1)
    if (e & 1) ret = mult(ret, b, m);
  return ret;
}
bool isPrime(u64 n) {  // determistic upto 7e^18
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  u64 A[] = {2, 325, 9375, 28178, 450775, 9780504,
      1795265022},
       s = __builtin_ctzll(n - 1), d = n >> s;
```

```
  for (u64 a : A) {
    u64 p = pw(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = mult(p, p, n);
    if (p != n - 1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd =
        q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) :=$ # of positive integers $\leq n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m,n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \ldots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \ldots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
$\sum_{d|n} \phi(d) = n$, $\sum_{1\leq k\leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a,n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p}\ \forall a$.

cf7d6d, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i
      ;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \mathrm{lcm}(m,n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

spf.h

Description: spf

1e547d, 20 lines

```
int MX = 1e7 + 1;
vi spf(MX + 1, INT32_MAX);
vector<int> is_prime(MX + 1, 1);
```

```
void sieve(int n = MX) {
  for (int i = 1; i <= n; i++)
    spf[i] = i;
  is_prime[0] = is_prime[1] = 0;
  int cnt = 1;
  for (int i = 2; i <= n; i++) {
    if (is_prime[i]) {
      for (int j = i * i; j <= n; j += i) {
        is_prime[j] = 0;
        spf[j] = min(i, spf[j]);
      }
      is_prime[i] = cnt;
      cnt++;
    }
  }
  return;
}
```

MatrixExpo.h

Description: MatrixExpo

32993d, 30 lines

```
int **matrixmul(int **matrix1, int **matrix2) {
  int **matrix3 = new int *[2];
  for (int i = 0; i < 2; i++)
    matrix3[i] = new int[2];

  matrix3[0][0] =
      (matrix1[0][0] * matrix2[0][0]) + (matrix1[0][1] *
         matrix2[1][0]);
  matrix3[0][1] =
      (matrix1[0][0] * matrix2[0][1]) + (matrix1[0][1] *
         matrix2[1][1]);
  matrix3[1][0] =
      (matrix1[1][0] * matrix2[0][0]) + (matrix1[1][1] *
         matrix2[1][0]);
  matrix3[1][1] =
      (matrix1[1][0] * matrix2[0][1]) + (matrix1[1][1] *
         matrix2[1][1]);
  matrix3[0][0] %= M;
  matrix3[1][0] %= M;
  matrix3[0][1] %= M;
  matrix3[1][1] %= M;

  return matrix3;
}

int **matrixexpo(int **matrix, int n, int **ans) {
  while (n > 0) {
    if (n % 2 == 1)
      ans = matrixmul(ans, matrix);
    matrix = matrixmul(matrix, matrix);
    n /= 2;
  }
  return ans;
}
```

BitBNS.h

Description: BitBNS

793511, 16 lines

```
// --- Bit Binary Search in o(log(n)) ---
const int M = 20;
const int N = 1 << M;

int lower_bound(int val)
{
  int ans = 0, sum = 0;
  for (int i = M - 1; i >= 0; i--)
  {
    int x = ans + (1 << i);
    if (sum + bit[x] < val)
      ans = x, sum += bit[x];
  }

  return ans + 1;
}
```

SomeDP.h

Description: SomeDP

ac57b7, 21 lines

```
// LIS
int lis(vector<int> const &a) {
  int n = a.size();
  const int INF = 1e9;
  vector<int> d(n + 1, INF);
  d[0] = -INF;

  for (int i = 0; i < n; i++) {
    int l = upper_bound(d.begin(), d.end(), a[i]) - d.
       begin();
    if (d[l - 1] < a[i] && a[i] < d[l])
      d[l] = a[i];
  }

  int ans = 0;
  for (int l = 0; l <= n; l++) {
    if (d[l] < INF)
      ans = l;
  }
  return ans;
}
// or segtree lol
```

sos.h

Description: sos dp

bc3748, 12 lines

```
vector<int> sos_dp(const vector<int>& A) {
    int n = __builtin_ctz(A.size());
    vector<int> F(A.begin(), A.end());
    for(int i = 0; i < n; ++i) {
        for(int mask = 0; mask < (1 << n); ++mask) {
            if(mask & (1 << i)) {
                F[mask] += F[mask ^ (1 << i)];
            }
        }
    }
    return F;
}
```

dnc.h

Description: dnc dp with cost func

d87640, 37 lines

```
auto add = [&](int v) {
    cnt[a[v]]++;
    ans += cnt[a[v]] - 1;
};
auto rem = [&](int v) {
    cnt[a[v]]--;
    ans -= cnt[a[v]];
};

auto getcost = [&](int expl, int expr) {
    while (l > expl) add(--l);
    while (l < expl) rem(l++);
    while (r < expr) add(++r);
    while (r > expr) rem(r--);
    return ans;
};

function<void(int, int, int, int)> calc = [&] (int l, int
     r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) / 2;
    array<int, 2> best = {LLONG_MAX, -1};
    for (int i = optl; i <= min(mid, optr); i++) {
        int inval = dp1[i] + getcost(i, mid);
        array<int, 2> cur = {inval, i};
        best = min(best, cur);
    }
    int opt = best[1];
    dp2[mid + 1] = best[0];
    calc(l, mid - 1, optl, opt);
    calc(mid + 1, r, opt, optr);
```

```
};
fr(i, 0, k) {
    calc(0, n-1, 0, n-1);
    dp1 = dp2;
    dp2 = vi(n+1, 1e15);
}
cout << dp1[n] << endl;
```

Numerical (6)

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j
        ;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

FastFourierTransform.h

Description: FastFourierTransform

00ced6, 37 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n);
    rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x
        : R[i / 2];
  }
  vi rev(n);
  rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
        C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
            rolled)
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i, 0, sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i, 0, sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

NTT.h

Description: NTT

f9d990, 52 lines

```
/* Description: Can be used for convolutions modulo
    specific nice primes of the form 2^a b+1, where the
    convolution result has size at most 2^a
 * (125000001 << 3) + 1 = 1e9 + 7, therefore do not use
    this for M = 1e9 + 7.
 * For $p < 2^30$ there is also e.g. (5 << 25, 3), (7 <<
    26, 3),
 * For other primes/integers, use two different primes
    and combine with CRT. (479 << 21, 3) and (483 << 21,
    5). The last two are > 10^9
 * Inputs must be in [0, mod).
 */
// Requires mod func
const int M = 998244353;
const int root = 3;
// (119 << 23) + 1, root = 3; // for M = 998244353
void ntt(int* x, int* temp, int* roots, int N, int skip)
    {
  if (N == 1) return;
  int n2 = N / 2;
  ntt(x, temp, roots, n2, skip * 2);
  ntt(x + skip, temp, roots, n2, skip * 2);
  for (int i = 0; i < N; i++) temp[i] = x[i * skip];
  for (int i = 0; i < n2; i++) {
    int s = temp[2 * i], t = temp[2 * i + 1] * roots[skip
        * i];
    x[skip * i] = (s + t) % M;
    x[skip * (i + n2)] = (s - t) % M;
  }
}
void ntt(vi& x, bool inv = false) {
  int e = pw(root, (M - 1) / sz(x));
  if (inv) e = pw(e, M - 2);
  vi roots(sz(x), 1), temp = roots;
  for (int i = 1; i < sz(x); i++) roots[i] = roots[i - 1]
      * e % M;
  ntt(&x[0], &temp[0], &roots[0], sz(x), 1);
}
// Usage: just pass the two coefficients list to get a*b
   (modulo M)
vi conv(vi a, vi b) {
  int s = sz(a) + sz(b) - 1;
  if (s <= 0) return {};
  int L = s > 1 ? 32 - __builtin_clzll(s - 1) : 0, n = 1
      << L;
  if (s <= 200) {  // (factor 10 optimization for |a|,|b|
      = 10)
    vi c(s);
    for (int i = 0; i < sz(a); i++)
      for (int j = 0; j < sz(b); j++)
        c[i + j] = (c[i + j] + a[i] * b[j]) % M;
    return c;
  }
  a.resize(n);
  ntt(a);
  b.resize(n);
  ntt(b);
  vi c(n);
  int d = pw(n, M - 2);
  for (int i = 0; i < n; i++) c[i] = a[i] * b[i] % M * d
      % M;
  ntt(c, true);
  c.resize(s);
  return c;
}
```

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
```

```
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b
        , b=c;
    a.pop_back();
  }
};
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0\ldots \geq n-1]$ and $tr[0\ldots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.

Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`

Time: $\mathcal{O}\left(n^2 \log k\right)$

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j])
          % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

Strings (7)

hash.h

Description: hash

52553f, 19 lines

```
template <int MOD, int P> struct RH {
  // using H1 = RH<1000000007, 91138233>;
  // using H2 = RH<1000000009, 97266353>;
  vector<long long> h, p;
  RH(const string &s) {
    int n = s.size();
    h.resize(n + 1, 0);
    p.resize(n + 1, 0);
    p[0] = 1;
    for (int i = 0; i < n; i++) {
      h[i + 1] = (h[i] * P + s[i]) % MOD;
      p[i + 1] = p[i] * P % MOD;
    }
  }
  long long get(int l, int r) { // [l,r]
    long long res = (h[r + 1] - h[l] * p[r - l + 1]) %
        MOD;
    return res < 0 ? res + MOD : res;
  }
```

```
};
```

kmp.h

Description: kmp

1c4d12, 28 lines

```
vector<int> prefix_function(string s) {
  int n = (int)s.length();
  vector<int> pi(n);
  for (int i = 1; i < n; i++) {
    int j = pi[i - 1];
    while (j > 0 && s[i] != s[j])
      j = pi[j - 1];
    if (s[i] == s[j])
      j++;
    pi[i] = j;
  }
  return pi;
}

vector<int> KMP(string text, string pattern) {
  string s = pattern + "#" + text;
  vector<int> pi = prefix_function(s);
  vector<int> matches;
  int p = pattern.length();

  for (int i = 0; i < s.length(); i++) {
    if (pi[i] == p) {
      int match_pos = i - 2 * p;
      matches.push_back(match_pos); // 0-based index in
          text
    }
  }
  return matches;
}
```

Trie.h

Description: Trie

009545, 77 lines

```
class TrieNode {
public:
  unordered_map<char, TrieNode *> children;
  bool isEndOfWord;

  TrieNode() : isEndOfWord(false) {}
};
class Trie {
private:
  TrieNode *root;

public:
  Trie() { root = new TrieNode(); }
  void insert(const string &word) {
    TrieNode *node = root;
    for (char ch : word) {
      if (node->children.find(ch) == node->children.end()
          ) {
        node->children[ch] = new TrieNode();
      }
      node = node->children[ch];
    }
    node->isEndOfWord = true;
  }
  bool search(const string &word) {
    TrieNode *node = root;
    for (char ch : word) {
      if (node->children.find(ch) == node->children.end()
          ) {
        return false;
      }
      node = node->children[ch];
    }
    return node->isEndOfWord;
  }
  bool startsWith(const string &prefix) {
    TrieNode *node = root;
```

```
    for (char ch : prefix) {
      if (node->children.find(ch) == node->children.end()
          ) {
        return false;
      }
      node = node->children[ch];
    }
    return true;
  }
};

// for bits
struct trieobject {
  trieobject() {
    children[0] = NULL;
    children[1] = NULL;
    numelems = 0;
  };

  struct trieobject* children[2];
  int numelems;
};

struct trie {
  trieobject base;
  trie() {
    trieobject base;
  }
  void add(int x) {
    int pow2 = (1ll << 31ll);
    trieobject* temp = &base;
    while (pow2 > 0) {
      if (temp->children[1 && (x & pow2)] == NULL) {
        temp->children[1 && (x & pow2)] = new trieobject;
      }
      temp->children[1 && (x & pow2)]->numelems++;
      temp = temp->children[1 && (x & pow2)];
      pow2 /= 2;
    }
  }
  // ADD FUNCTION BELOW

};
```

Manacher.h

Description: Manacher

cd719d, 16 lines

```
/* Description: p[0][i] = half length of longest even
    palindrome behind pos i,
p[1][i] = longest odd with center at pos i(half rounded
    down). */
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi, 2> p = {vi(n + 1), vi(n)};
  for (int z = 0; z < 2; z++)
    for (int i = 0, l = 0, r = 0; i < n; i++) {
      int t = r - i + !z;
      if (i < r) p[z][i] = min(t, p[z][l + t]);
      int L = i - p[z][i], R = i + p[z][i] - !z;
      while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
        p[z][i]++, L--, R++;
      if (R > r) l = L, r = R;
    }
  return p;
}
```

SuffixArray.h

Description: SuffixArray

c54b5a, 32 lines

```
/*Builds suffix array for a string.
\texttt{sa[i]} is the starting index of the suffix which
is $i$'th in the sorted suffix array.
The returned vector is of size $n+1$, and \texttt{sa[0] =
    n}.
```

```
The \texttt{lcp} array contains longest common prefixes
    for
neighbouring strings in the suffix array:
\texttt{lcp[i] = lcp(sa[i], sa[i-1])}, \texttt{lcp[0] = 0
    }.
The input string must not contain any zero bytes.
Time: O(n \log n)*/
#define rep(i, j, k) for (int i = j; i < k; i++)
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string& s, int lim = 256) {  // or
      basic_string<int>
    int n = sz(s) + 1, k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    x.push_back(0), sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim
        = p) {
      p = j, iota(all(y), n - j);
      rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i, 0, n) ws[x[i]]++;
      rep(i, 1, lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i, 1, n) a = sa[i - 1], b = sa[i], x[b] = (y[a]
          == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k &&k--, j = sa[x[i] - 1];
           s[i + k] == s[j + k]; k++)
        ;
  }
};
```

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x <
    0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p
      .y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p
      .y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*
      this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()
      =1
  P perp() const { return P(-y, x); } // rotates +90
      degrees
  P normal() const { return perp().unit(); }
```

```
  // returns point rotated 'a' radians ccw around the
     origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: `vector<Angle> v = {w[0], w[0].t360() ...}; // sorted int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; } // sweeps j such that (j-i) represents the number of positively oriented // triangles with vertices at 0 and i`

0f0602, 35 lines

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y,
      t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half() && x >=
      0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare
     distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle
   between
// them, i.e., the angle that covers the defined line
   segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b <
      a)};
}
```

OnSegment.h

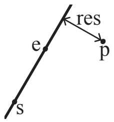
Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

LineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

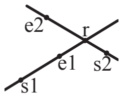


"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

LineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: auto res = lineInter(s1,e1,s2,e2);
```
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
```

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

LineProjectionReflection.h

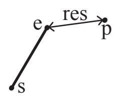
Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.



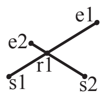
Usage: Point<double> a, b(2,2), p(1,1);
```
bool onSegment = segDist(a,b,p) < 1e-10;
```

"Point.h" 5c88f4, 6 lines

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)
      ));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



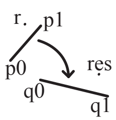
Usage:
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d)
    {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

LinearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



"Point.h" 03a306, 6 lines

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq))
     ;
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.
     dist2();
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>*
    out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*
           p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2)
     / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" e0cfba, 9 lines

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
```

```
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double
    r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q)
        > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

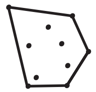
Time: $\mathcal{O}(n)$

"Point.h" 9706dc, 9 lines

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0)
        t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h
     [1])};
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true)
   {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p)
     ;
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)
     <= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p
       + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h" bac5b0, 63 lines

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0;

  T distance(const P& p) { // min squared distance to a
     point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
      // divide by taking half the array for each child (
         not
      // best performance with many duplicates in the
         middle)
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});
    }
  }
};
```

```
struct KDTree {
  Node* root;
  KDTree(const vector<P>& vp) : root(new Node({all(vp)}))
      {}

  pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
      // uncomment if we should not find the point itself
         :
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    }

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }

  // find nearest point to a point, and its squared
     distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(const P& p) {
    return search(root, p);
  }
};
```